

MemoTime: Memory-Augmented Temporal Knowledge Graph Enhanced Large Language Model Reasoning

Xingyu Tan
UNSW
Data61, CSIRO
Sydney, Australia
xingyu.tan@unsw.edu.au

Xiaoyang Wang*
UNSW
Sydney, Australia
xiaoyang.wang1@unsw.edu.au

Qing Liu
Data61, CSIRO
Hobart, Australia
q.liu@data61.csiro.au

Xiwei Xu
Data61, CSIRO
Sydney, Australia
xiwei.xu@data61.csiro.au

Xin Yuan
Data61, CSIRO
UNSW
Sydney, Australia
xin.yuan@data61.csiro.au

Liming Zhu
Data61, CSIRO
Sydney, Australia
liming.zhu@data61.csiro.au

Wenjie Zhang
UNSW
Sydney, Australia
wenjie.zhang@unsw.edu.au

Abstract

Large Language Models (LLMs) have achieved impressive reasoning abilities, but struggle with temporal understanding, especially when questions involve multiple entities, compound operators, and evolving event sequences. Temporal Knowledge Graphs (TKGs), which capture vast amounts of temporal facts in a structured format, offer a reliable source for temporal reasoning. However, existing TKG-based LLM reasoning methods still struggle with four major challenges: maintaining temporal faithfulness in multi-hop reasoning, achieving multi-entity temporal synchronization, adapting retrieval to diverse temporal operators, and reusing prior reasoning experience for stability and efficiency. To address these issues, we propose **MemoTime**, a memory-augmented temporal knowledge graph framework that enhances LLM reasoning through structured grounding, recursive reasoning, and continual experience learning. MemoTime decomposes complex temporal questions into a hierarchical Tree of Time, enabling operator-aware reasoning that enforces monotonic timestamps and co-constrains multiple entities under unified temporal bounds. A dynamic evidence retrieval layer adaptively selects operator-specific retrieval strategies, while a self-evolving experience memory stores verified reasoning traces, toolkit decisions, and sub-question embeddings for cross-type reuse. Comprehensive experiments on multiple temporal QA benchmarks show that MemoTime achieves overall state-of-the-art results, outperforming the strong baseline by up to 24.0%. Furthermore, MemoTime enables smaller models (e.g., Qwen3-4B) to achieve reasoning performance comparable to that of GPT-4-Turbo.

CCS Concepts

• Information systems → Question answering.

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792581>

Keywords

Large Language Models; Retrieval-Augmented Generation; Temporal Knowledge Graph; Temporal Knowledge Graph Question Answering; Memory-Augmented Retrieval-Augmented Generation

ACM Reference Format:

Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, Liming Zhu, and Wenjie Zhang. 2026. MemoTime: Memory-Augmented Temporal Knowledge Graph Enhanced Large Language Model Reasoning. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792581>

1 Introduction

Large Language Models (LLMs) have demonstrated strong performance across a wide range of tasks by scaling to billions of parameters and pre-training on massive and diverse text corpora [2]. However, due to the prohibitive cost of retraining, these models are inherently static. This limitation results in factual gaps, temporal inconsistencies, and reasoning hallucinations when LLMs are required to process time-sensitive or evolving information [20, 33, 45].

Retrieval-Augmented Generation (RAG) has emerged as an effective paradigm to mitigate these limitations by allowing LLMs to access external information sources during inference [14, 15, 69]. Typical RAG systems embed both questions and documents into a shared vector space and retrieve semantically similar passages [27, 48]. Although effective in many cases, such retrieval often prioritizes semantic similarity while neglecting structural and temporal dependencies across entities and events [17]. For instance, two temporally distinct statements such as “Barack Obama is the President of the United States (2009–2017)” and “Joe Biden is the President of the United States (2021–)” may appear semantically related, yet they cannot simultaneously hold true. Plain-text retrieval often fails to distinguish between such temporally exclusive facts, leading to incorrect or contradictory reasoning chains.

Temporal reasoning further amplifies these challenges. Real-world entities and relationships evolve continuously, i.e., people change roles, organizations merge, and events unfold in sequence over time [4, 24]. Temporal-aware questions therefore often involve implicit time constraints, multiple dependent conditions, and mixed temporal granularities such as “Who chaired the committee

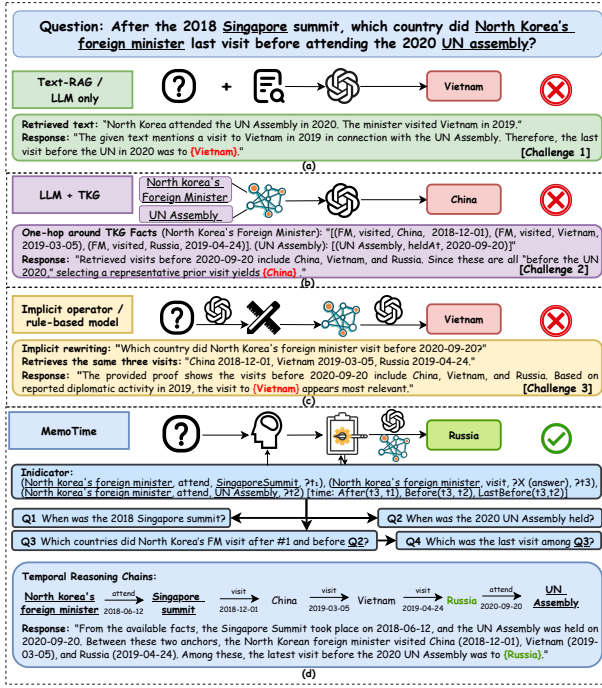


Figure 1: Representative workflow of four LLM reasoning paradigms.

before the 2010 reform?" or "Which country hosted the first summit after 2015?". These questions require reasoning that integrates both temporal and semantic alignment, which remains difficult for text-based RAG methods that lack structured temporal awareness.

To address this issue, integrating Temporal Knowledge Graphs (TKGs) with LLM reasoning provides a promising direction [4, 13, 17, 20, 35]. TKGs encode factual knowledge as quadruples (subject, relation, object, timestamp), offering explicit temporal grounding and relational structure. By leveraging TKGs, models can reason over evolving entities while maintaining factual and temporal consistency. Temporal Knowledge Graph Question Answering (TKGQA) serves as a representative evaluation for this task, requiring systems to answer natural language questions by retrieving temporally relevant facts from TKGs.

Challenges in existing methods. Most existing TKG-based reasoning frameworks follow a "plan-retrieve-answer" pipeline. In this paradigm, LLMs decompose complex temporal questions into a series of sub-tasks, retrieve related facts from a TKG, and generate an answer based on the retrieved context. While this improves interpretability and modularity, several challenges face.

Challenge 1: Temporal faithfulness in multi-hop reasoning. Most temporal QA pipelines [4, 17, 35], as shown in Figure 1(a), prioritize semantic similarity over chronological accuracy. Expanding from one-hop neighbors and ranking by semantics often retrieves paths that appear contextually relevant but violate time constraints. For example, answering "Which country did X last visit before 2020?" may simply return (X, visit, Y, 2019) because of lexical overlap, despite contradicting the before-last relation. In multi-hop queries such as "Who did X hire after Y resigned during Q3 2017?", locally valid hops can combine into a globally inconsistent reasoning chain.

Challenge 2: Multi-entity temporal synchronization. When a question contains multiple entities, most systems [3, 35, 43] explore each entity independently and attempt to merge partial evidence later. This disjoint process often produces candidates that never align within a single, time-consistent reasoning path. For instance, as shown in Figure 1(b), independent exploration can yield valid facts for each entity, but fails to synchronize their temporal windows.

Challenge 3: Operator diversity and adaptive retrieval. Temporal questions encompass diverse operators, each requiring a distinct reasoning policy. Previous methods [35, 36] focus on single-round rewriting and can expose explicit timestamps but struggle with combined constraints, as shown in Figure 1(c). It prevents the model from adapting to such heterogeneity, resulting in either under-coverage (missing valid evidence) or over-retrieval (noisy results).

Challenge 4: Lack of reasoning experience management. Most existing pipelines [13, 27, 35, 47] remain memoryless, discarding successful reasoning traces after each run. They often rely on manually crafted or static exemplars, which are costly to construct and insufficiently generalizable across diverse temporal question types, leading models repeatedly re-solve similar sub-questions from scratch, failing to transfer prior knowledge across operators or tasks.

Contribution. In this paper, we introduce **MemoTime**, a Memory-Augmented Temporal Knowledge Graph framework designed to enhance LLM temporal reasoning, as shown in Figure 1(d). Unlike existing methods that rely on static retrieval or pre-defined templates, MemoTime integrates structured temporal grounding, hierarchical reasoning, dynamic toolkit invocation, and continual memory updating into a unified framework. It enables LLMs to reason faithfully over time-aware facts, adapt retrieval strategies to temporal operators, and progressively improve performance through experience reuse.

To address multi-hop temporal reasoning problem, MemoTime introduces a hierarchical reasoning framework that decomposes complex temporal questions under a unified global plan. All sub-indicators are evolved from the root indicator of the main question, ensuring that each branch inherits consistent temporal constraints and preserves monotonic timestamp progression. Guided by this global supervision, MemoTime performs controlled branch expansion, retrieving multi-hop reasoning paths remaining faithful to both semantic relevance and chronological order.

To handle multi-entity temporal synchronization, MemoTime ensures that the final reasoning path jointly incorporates all topic entities under a unified temporal framework and synchronized timeline. Instead of exploring entities independently, the system retrieves co-constrained evidence paths that preserve shared temporal bounds and consistent granularity, ensuring global coherence and containing the factual basis for the correct answer.

To adapt to diverse temporal operators, MemoTime introduces a library of temporal reasoning toolkits that support heterogeneous operators. Instead of fixed templates, MemoTime adaptively selects the most suitable toolkit through experience-guided prompts. This adaptive retrieval policy ensures that each temporal operator triggers an appropriate reasoning strategy, thereby improving precision and reducing over-retrieval noise.

To manage and reuse reasoning experience, MemoTime maintains a continuously evolving experience memory that records successful records. Each experience entry is stored alongside embeddings of both the question and its temporal indicator, enabling efficient similarity-based retrieval under operator and type constraints. During inference, MemoTime retrieves relevant exemplars to guide new reasoning, while after execution, verified trajectories are written back. The advantage of MemoTime can be abbreviated as follows:

- **Memory-augmented temporal reasoning.** MemoTime introduces a unified framework that integrates dynamic memory retrieval and update into temporal-aware question decomposition, allowing the model to recall, reuse, and refine reasoning trajectories for long-term temporal understanding.
- **Hierarchical and interpretable control.** A hierarchical controller Tree of Time executes, verifies, and refines sub-questions, ensuring faithful and interpretable temporal reasoning.
- **Hybrid retrieval and pruning.** An operator-aware retrieval layer combines symbolic graph expansion with embedding search, applying temporal-first pruning and semantic re-ranking for both chronological validity and precise evidence grounding.
- **Self-evolving experience memory.** Verified reasoning responses are continuously organized in an adaptive experience pool, forming a closed feedback loop for continual improvement.
- **Efficiency and adaptability:** a) MemoTime is a plug-and-play framework that can be seamlessly applied to various LLMs and TKGs. b) MemoTime is auto-refresh. New information is incorporated instantly via TKG retrieval instead of costly LLM fine-tuning. c) MemoTime achieves state-of-the-art results on all the tested datasets, surpasses the strong baseline by up to 24.0%, and enables smaller models (e.g., Qwen3-4B) to achieve reasoning performance comparable to GPT-4-Turbo.

2 Related Work

LLM-based knowledge graphs reasoning. Graphs are a natural representation for modeling relational structure among entities [22, 23, 46, 51–55, 63, 64]. Knowledge graphs (KGs) provide structured, verifiable knowledge that complements the implicit world knowledge in LLMs [12, 32, 57–59]. Early works embedded KG facts into neural networks during pre-training or fine-tuning [26, 39, 40, 65, 68], but such approaches hinder efficient updates and reduce interpretability. Recent studies explore LLM–KG integration by prompting LLMs to iteratively traverse graphs, as seen in [19, 43, 49, 62]. These systems guide the LLM to expand reasoning paths from a seed entity and refine answers through repeated retrieval–generation cycles. However, starting from a single vertex overlooks multi-entity connections and temporal dependencies, often yielding semantically plausible yet chronologically inconsistent paths. [47] alleviates this by modeling multi-hop reasoning paths, but relies on static KGs and lacks the dynamic temporal alignment.

Temporal knowledge graphs question answering. Temporal Knowledge graphs extend static graphs with timestamped facts, enabling reasoning over evolving entities and relations [60, 66, 67]. Earlier approaches fall into two main categories: semantic parsing-based and embedding-based methods. Parsing-based methods translate natural-language questions into logical forms executable on TKGs [7, 16, 31]. These achieve precise execution but fail on long

or ambiguous queries due to brittle symbolic parsing. Embedding-based methods [28, 28, 36] learn vectorized temporal reasoning by aligning question embeddings with fact embeddings, but they are limited to short reasoning chains and simple time expressions. Recent LLM-based systems [11, 17, 35] improve interpretability by generating reasoning steps in natural language.

Hierarchical and memory-augmented reasoning. Recent advances in LLM-based QA frameworks have emphasized question decomposition as a way to improve reasoning depth and interpretability. Multi-hop reasoning systems such as [61] decompose complex queries into simpler sub-questions that are independently solved and aggregated. This decomposition enables finer-grained retrieval and targeted reasoning across heterogeneous evidence sources [10]. However, these methods decompose linearly, leading to accumulated errors. Moreover, most existing decomposition modules are either manually designed [30] or fine-tuned [56] for specific datasets, limiting their adaptability across temporal and structural question types. In addition to the decomposition, recent studies explore long-term memory systems for LLM agents [29]. Approaches such as [70] enable retrieval of prior interaction histories, while SCM [50] maintains selective access through controller mechanisms. But these frameworks are typically task-agnostic and lack structured representations of reasoning processes.

3 Preliminaries

Consider a Temporal Knowledge Graph (TKG) $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$, where $\mathcal{E}$, $\mathcal{R}$, and $\mathcal{T}$ represent the set of entities, relations, and timestamps, respectively. $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$ contains abundant temporal factual knowledge in the form of quadruples, i.e., $\mathcal{G} = \{(e_h, r, e_t, t) \mid e_h, e_t \in \mathcal{E}, r \in \mathcal{R}, t \in \mathcal{T}\}$. Temporal constraint defines a condition related to a specific time point or interval that must be satisfied by both the answer and its supporting evidence. Following Allen’s interval algebra [1], this includes the 13 classical temporal relations, as well as extended operators such as temporal set relations, duration comparisons, and sorting mechanisms (e.g., first, last, nth) [44]. Temporal constraints thus enable fine-grained alignment between natural language queries and temporally consistent evidence paths.

DEFINITION 1 (TEMPORAL PATH). Given a TKG $\mathcal{G}$, a temporal path is a connected sequence of temporal facts, represented as: $\text{path}_{\mathcal{G}}(e_1, e_{l+1}) = \{(e_1, r_1, e_2, t_1), (e_2, r_2, e_3, t_2), \dots, (e_l, r_l, e_{l+1}, t_l)\}$, where l denotes the length of the path, i.e., $\text{length}(\text{path}_{\mathcal{G}}(e_1, e_{l+1})) = l$. The path must satisfy two constraints: (1) connectivity: the tail entity of each fact is identical to the head entity of the next; (2) temporal monotonicity: timestamps are non-decreasing, i.e., $t_1 \leq t_2 \leq \dots \leq t_l$.

EXAMPLE 1 (TEMPORAL PATH). Consider a temporal path between “Merkel” and “EU” with a length of 3: $\text{path}_{\mathcal{G}}(\text{Merkel}, \text{EU}) = \{(\text{Merkel}, \text{visit}, \text{Paris}, 2012), (\text{Paris}, \text{host}, \text{Conference}, 2013), (\text{Conference}, \text{attended_by}, \text{EU}, 2014)\}$, and can be visualized as:

$$\text{Merkel} \xrightarrow[2012]{\text{visit}} \text{Paris} \xrightarrow[2013]{\text{host}} \text{Conference} \xrightarrow[2014]{\text{attended_by}} \text{EU}.$$

DEFINITION 2 (TEMPORAL REASONING PATH). Given a TKG $\mathcal{G}$, and an entity list $[e_1, e_2, \dots, e_n]$, a temporal reasoning path is a sequence of temporal segments $\text{TRP}_{\mathcal{G}}([e_1, \dots, e_n]) = \{P_1, P_2, \dots, P_n\}$, where each $P_i = \text{path}_{\mathcal{G}}(e_i, n_i)$ for some $n_i \in \mathcal{E}$, and the global monotonicity condition holds: $t^{\text{end}}(P_i) \leq t^{\text{start}}(P_{i+1})$, $\forall 1 \leq i < n$.

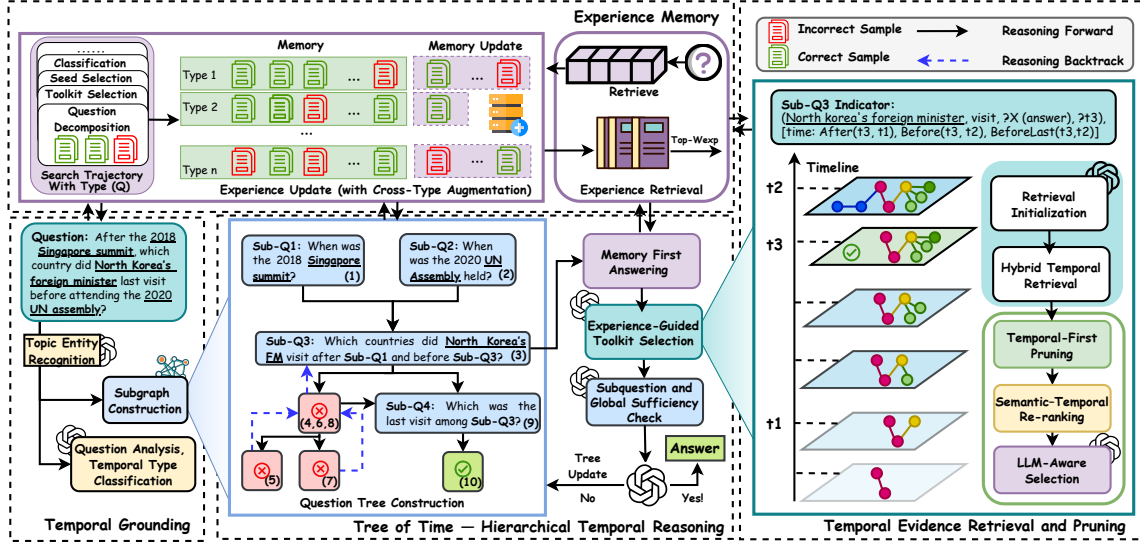


Figure 2: Overview of the MemoTime framework. **Temporal Grounding:** Topic entities and temporal operators are extracted from the input question to construct a question-specific subgraph. **Tree of Time (Hierarchical Reasoning):** Recursively decomposes the question into sub-questions guided by temporal dependencies, adaptively reusing experience or invoking toolkits for new evidence. **Temporal Evidence Retrieval and Pruning:** Performs operator-aware retrieval under monotonic time constraints, followed by semantic-temporal re-ranking and LLM-based sufficiency verification. **Experience Memory:** Verified reasoning traces are stored, updated, and retrieved for cross-type reuse, enabling continual self-improvement across reasoning cycles.

EXAMPLE 2 (TEMPORAL REASONING PATH). Consider a temporal reasoning path $TRP_G([Obama, Beijing, Paris])$, by two temporally aligned segments: $P_1 = \{(Obama, meet, UN, 2009)\}$, $P_2 = \{(Beijing, linked_via, EU, 2011), (EU, event_in, Paris, 2012)\}$, visualized as:

$$Obama \xrightarrow[2009]{meet} UN \rightsquigarrow Beijing \xrightarrow[2011]{linked_via} EU \xrightarrow[2012]{event_in} Paris.$$

Temporal Knowledge Graph Question Answering (TKGQA) is a fundamental reasoning task based on TKGs. Given a natural language question Q and a TKG $\mathcal{G}$, the objective is to devise a function f that identify answer entities or timestamps $a \in Answer(Q)$ utilizing knowledge in $\mathcal{G}$, i.e., $a = f(q, \mathcal{G})$. Consistent with previous research [26, 27, 42, 43], we assume topic entities $Topics$ are mentioned in Q and answer entities or timestamps $Answer(q)$ in ground truth are linked to $\mathcal{G}$, i.e., $Topics \subseteq \mathcal{E}$ and $Answer(Q) \subseteq \{\mathcal{E} \cup \mathcal{T}\}$.

4 Method

Overview. MemoTime implements “TKG-grounded LLM reasoning” by grounding each question in temporal facts, recursively decomposing it into executable sub-queries, retrieving and pruning temporally valid evidence, and continually refining its experience memory. Unlike prior TKGQA or RAG systems that depend on static retrievers or fixed prompts, MemoTime integrates temporal alignment, hierarchical control, and self-evolving memory into a unified reasoning framework. The overall architecture of MemoTime is detailed in Figure 2, consisting of four key components:

- **Temporal Grounding.** The model begins by linking topic entities and constructing a $D_{\max}$ -hop temporal subgraph $\mathcal{G}_Q$ from the TKG. It then classifies the temporal type of the question using exemplars retrieved from memory, producing structured temporal constraints for downstream reasoning (Section 4.1).

- **Tree of Time Reasoning.** Given the grounded question and its temporal type, MemoTime constructs a hierarchical decomposition tree and executes sub-questions in a top-down manner. For each node, it dynamically decides whether to recall prior experiences, invoke temporal toolkits, or refine unresolved branches, ensuring temporal consistency and interpretability (Section 4.2).
- **Temporal Evidence Retrieval and Pruning.** Guided by toolkit configurations, MemoTime performs hybrid retrieval, combining time-monotone, graph exploration, and embedding-based search. Retrieved candidates are filtered by temporal constraints, re-ranked by semantic and temporal proximity, and finally verified through an LLM-aware selection step (Section 4.3).
- **Experience Memory.** Verified reasoning traces, toolkit selections, and sub-question embeddings are stored in a dynamic memory pool. At inference, similar experiences are retrieved by type and relevance; after reasoning, new traces are written back to continuously refine retrieval and decision-making across future questions (Section 4.4).

4.1 Temporal Grounding

The temporal grounding stage transforms a natural-language temporal question into a structured reasoning representation, establishing a factual and temporal foundation for subsequent retrieval and inference. It consists of two core phases: knowledge fact grounding and question analysis. The pseudo-code of the temporal grounding is detailed in Algorithm 1 of Appendix A.

Knowledge fact grounding. Given an input question Q , MemoTime first identifies the relevant temporal subgraph $\mathcal{G}_Q$ within the underlying TKG. This subgraph captures entities, relations, and time-stamped triples that are semantically or temporally associated

with Q within a bounded $D_{\max}$ -hop neighborhood, serving as the factual evidence base for downstream reasoning.

Topic entity recognition. To locate question-relevant entities, MemoTime employs LLMs to extract potential entity mentions and associated timestamps from Q . Following extraction, a Dense Retrieval Model (DRM) aligns these mentions with KG entities via embedding-based similarity matching. Specifically, both question keywords and KG entities are encoded into dense embeddings, and an entity index is constructed using FAISS [9]. Cosine similarity is then computed between the two embedding spaces, and the top-ranked entities are selected to form the topic entity set $Topics$. These topic entities act as the initial anchors for constructing a localized subgraph.

Subgraph construction. Once the topic entities are identified, MemoTime constructs a $D_{\max}$ -hop subgraph $\mathcal{G}_Q$ that aggregates all triples connected to each topic entity within the defined temporal window. This subgraph provides a compact yet semantically rich neighborhood for temporal reasoning. To reduce redundancy and computational overhead, we apply graph reduction and relation clustering techniques following [47], ensuring $\mathcal{G}_Q$ remains concise while preserving the most relevant temporal relations.

Question analysis. After constructing $\mathcal{G}_Q$, the next step is to analyze the temporal intent of the question. Unlike conventional TKGQA approaches that treat all questions uniformly, or use the label from the dataset, leveraging the flexibility reduces. Temporal questions exhibit diverse syntactic forms and intricate time constraints. To capture these nuances, MemoTime first performs temporal type classification, identifying the temporal operator that governs the question’s reasoning logic.

Temporal type classification. Temporal type classification determines the intrinsic temporal relations implied by a question. Following Allen’s interval algebra [1], we consider thirteen fundamental temporal relations, as well as extended operators such as set relations, duration comparisons, and ordering mechanisms [44], as introduced in Section 3. Existing methods often rely on static prompts or manually designed exemplars, limiting their adaptability to complex or unseen temporal structures. In contrast, MemoTime leverages a continuously updated experience pool $\mathcal{E}_{pool}$ that stores successful reasoning trajectories, toolkit configurations, and classification results. This memory-based design enables dynamic retrieval of contextually aligned exemplars, improving temporal reasoning consistency across question types. Formally, for a given question Q , type-specific exemplars are retrieved as: $\mathcal{E}_{type} = \text{GetTypeExp}(\mathcal{E}_{pool}, Q, W_{exp})$, where W_{exp} denotes the exemplar retrieval limit. These exemplars are then incorporated into the LLM classification prompt: $\text{Type}(Q) = \text{Prompt}_{\text{TypeSelect}}(Q, \mathcal{E}_{type})$, allowing the model to identify the most appropriate temporal operator for Q . This exemplar-guided strategy enables the MemoTime to adapt to new question patterns, enhances type prediction accuracy, and lays the foundation for the subsequent decomposition stage.

4.2 Tree of Time — Hierarchical Temporal Reasoning

After temporal grounding, MemoTime enters the Tree of Time (ToT) stage, which transforms grounded temporal structures into a hierarchical reasoning tree. Serving as the framework’s central

controller, it coordinates decomposition, toolkit execution, sufficiency testing, and answer synthesis. Guided by the global plan, the model adaptively alternates between recalling experience, invoking operator-specific toolkits, and refining sub-questions to maintain temporal and semantic consistency. Due to the space limitation, the pseudo-code of the ToT is detailed in Algorithm 2 of Appendix A.

Question tree construction. Given a temporal question Q and its classified temporal type $\text{Type}(Q)$, MemoTime begins by constructing a hierarchical decomposition tree $\mathcal{T}_Q$. Each node represents a sub-question with a specific temporal relation or constraint. To enhance robustness, decomposition exemplars are retrieved from the experience pool $\mathcal{E}_{pool}$ and integrated into the LLM prompt:

$$\mathcal{E}_{decomp} = \text{GetDecompExp}(\mathcal{E}_{pool}, Q, W_{exp}, \text{Type}(Q)),$$

$$\mathcal{T}_Q = \text{Prompt}_{\text{Decompose}}(Q, \mathcal{E}_{decomp}, \text{Type}(Q)).$$

This experience-guided process enables the system to build a type-aligned reasoning tree that reflects temporal dependencies among sub-questions. For any parent-child pair (q_i, q_j) , the temporal order satisfies $t(q_i) \leq t(q_j)$, ensuring temporal monotonicity.

Indicator extraction. From $\mathcal{T}_Q$, the system extracts a set of sub-question indicators $\{I_i\}$, i.e., $I_i = \langle x?, R, y?, C_{time} \rangle$, each encoding the entities, relations, and temporal constraints required for reasoning. Based on this, a predicted depth $D_{pred}(q_i)$ is calculated, defined as the maximum distance between the predicted answer and each topic entity. For instance, in Figure 1(d), the predicted depth of the overall indicator is 2.

Hierarchical execution control. The reasoning tree is traversed in a top-down manner. At each node, MemoTime first checks whether a similar reasoning trace exists in the experience pool for direct reuse. If no match or evidence insufficient, the system proceeds to dynamic toolkit selection to discover new evidence.

Experience-guided toolkit selection. When memory lookup fails, MemoTime retrieves prior examples of successful toolkit usage to enhance selection accuracy:

$$\mathcal{E}_{tool} = \text{GetToolkitExp}(\mathcal{E}_{pool}, q_i, I_i, \text{Type}(Q), W_{exp}),$$

and constructs a context-enriched prompt: $\mathcal{T}_i = \text{Prompt}_{\text{ToolkitSelect}}(\mathcal{T}_Q, I_i, q_i, \mathcal{E}_{tool})$. The LLM may recommend multiple toolkits simultaneously, each representing a distinct retrieval or reasoning strategy (e.g., event ordering, interval comparison, or timeline construction). All selected tools are executed in parallel, and their candidate results are passed to the sufficiency evaluation stage.

Question answering. After each sub-question q_i obtains its candidate temporal results, MemoTime evaluates whether these results sufficiently answer the local query and contribute to the overall question. This stage integrates evidence evaluation, global reasoning synthesis, and hierarchical update for unresolved nodes.

Sub-question evaluation. For each node in the reasoning tree, the LLM assesses retrieved temporal paths for both semantic relevance and temporal validity. A sub-question is considered sufficient when one or more candidate paths provide consistent evidence aligned with its indicator. If multiple valid results exist, a lightweight debate-vote strategy aggregates them into a single coherent answer. The solved node and its reasoning trace are recorded in the experience pool $\mathcal{E}_{pool}$ for future reuse.

Global sufficiency and answer synthesis. After all sub-questions are processed, MemoTime checks the temporal and logical coherence of the entire reasoning tree. When global sufficiency is achieved, verified sub-paths are merged into a unified temporal reasoning chain that summarizes the full evidence flow. This concise reasoning trace is then used to prompt the final LLM for answer generation, ensuring interpretability and temporal consistency.

Tree update and termination. If a sub-question fails sufficiency verification, MemoTime adaptively decides whether to retry retrieval, refine the query, or perform further decomposition based on its current reasoning. Newly generated nodes are appended to the tree and processed recursively until all branches are solved or the maximum depth $D_{\max}$ is reached. Finally, verified reasoning paths are consolidated into the final evidence chain for answer generation.

4.3 Temporal Evidence Retrieval and Pruning

As discussed in Section 1, identifying reasoning paths that connect all topic entities is crucial for deriving accurate answers. These paths function as interpretable chains of thought. In this section, we operationalize the selected toolkits to perform hybrid retrieval and pruning, balancing efficiency and accuracy while ensuring both high temporal alignment and broad semantic coverage. The pseudo-code of this section is detailed in Algorithm 3 (AppendixA).

Retrieval initialization. Before path exploration begins, MemoTime performs seed selection to identify relevant entities that serve as starting points for reasoning. If prior successful seeds exist in $\mathcal{E}_{pool}$, they are reused directly; otherwise, new seeds are chosen via the experience-augmented prompt: $S_i = \text{Prompt}_{\text{SeedSelect}}(\text{Topics}, I_i, q_i, \mathcal{E}_{\text{seed}})$, where $\mathcal{E}_{\text{seed}}$ denotes the retrieved relevant examples from memory.

Toolkit-driven temporal path retrieval. Each toolkit $T_\theta \in \mathcal{T}_i$ represents a specialized retrieval operator for a temporal reasoning pattern (e.g., one-hop expansion, interval comparison, event ordering, or range detection). Guided by the indicator I_i , each toolkit constrains both the semantic relation and temporal boundary of the retrieval. Multiple toolkits are executed in parallel to form a diverse set of candidate paths $\mathcal{P}_i$.

Hybrid temporal retrieval. For graph-based retrieval, the system performs time-monotonic path expansion. Instead of starting from the maximum depth $D_{\max}$, MemoTime begins exploration at the predicted depth $D_{\text{pred}}(I_i)$. Given the subgraph $\mathcal{G}_O$, the ordered seed set S_i , the indicator I_i , and the depth $D = \min(D_{\text{pred}}(I_i), D_{\max})$, we identify candidate temporal paths that include all seeds in order. To avoid exhaustive search, we apply a tree-structured bidirectional breadth-first search (BiBFS) from each entity to extract all potential paths, defined as: $C = \{p \mid |S_i| \cdot (D-1) < \text{length}(p) \leq |S_i| \cdot D\}$. In parallel, an embedding retriever retrieves semantically relevant facts from document-indexed KG segments, ensuring that implicit or cross-hop relations are also captured. The two result streams are concatenated and unified into a hybrid candidate path pool.

Temporal and semantic pruning. Candidate paths are then filtered through a multi-stage pruning pipeline by temporal consistency, semantics, and LLM selection. Together, these steps drastically reduce noise while retaining only candidates that satisfy both chronological and semantic alignment.

Temporal-first pruning. Traditional relevance-first pruning may retain semantically similar but temporally inconsistent paths. To mitigate this, MemoTime adopts a *temporal-first* pruning policy. Given the candidate pool $C = \{p_i\}_{i=1}^N$ returned by the hybrid retriever, MemoTime first applies a strict temporal filter that removes paths violating time constriction C_{time} or non-monotonic progressions, yielding a valid subset $\tilde{C} \subseteq C$ that satisfies all temporal constraints. **Semantic-temporal re-ranking.** From $\tilde{C}$, candidates are re-ranked by jointly considering (i) semantic compatibility with the indicator and (ii) proximity of their timestamps to the expected reference time. Let $\text{DRM}(I_i, p)$ denote the semantic similarity between the indicator and path, and $t(\cdot)$ denote the representative timestamp. Each candidate receives a composite score:

$$\text{Score}(p) = \lambda_{\text{sem}} \cdot \text{DRM}(I_i, p) + \lambda_{\text{prox}} \cdot \exp(-|t(p) - t(I_i)|/\sigma),$$

where λ_{sem} and λ_{prox} balance semantic alignment and temporal proximity. The top- W_1 candidates are retained as a compact, high-quality pool for LLM-based selection.

LLM-aware selection. Following the re-ranking procedure, the candidate paths are reduced to W_1 . We then prompt LLM to score and select the top- $W_{\max}$ reasoning paths most likely to satisfy question's temporal and semantic constraints. This final step emphasizes faithfulness and interpretability, producing a minimal but high-precision evidence set for the sufficiency verification in the reasoning loop.

4.4 Experience Memory

After each reasoning episode, verified temporal paths and toolkit decisions are stored for future reuse, forming the foundation of the experience memory layer. This layer serves as the long-term knowledge base of MemoTime, recording reasoning trajectories, toolkit selection patterns, and verified temporal facts. It bridges different reasoning cycles by allowing the model to retrieve relevant exemplars and continually refine its decision-making through accumulated experience. Unlike static prompt libraries, the memory operates as a dynamic, self-evolving component supporting both retrieval and continual adaptation across diverse question types. The pseudo-code of the section is shown in Algorithm 4 of Appendix A.

Experience retrieval. During the Tree of Time reasoning process, each sub-task may query the experience pool $\mathcal{E}_{pool}$ to obtain relevant exemplars for decomposition, seed selection, or toolkit usage. All retrieval operations (e.g., GetTypeExp, GetDecompExp, GetToolkitExp) are implemented under a unified interface: $\mathcal{E} = \text{RetrieveExperience}(\mathcal{E}_{pool}, q_i, I_i, \tau, W_{\text{exp}})$, where $\tau = \text{Type}(Q)$ denotes the temporal type of the current reasoning context. Each query is restricted to the memory subset that shares the same type label τ , ensuring contextual consistency and preventing cross-type noise. Every experience record stores both textual metadata and its dense embeddings, one for the question text and another for its indicator, allowing semantic-temporal similarity search through a FAISS-based index [9]. The incorrect samples are used as warnings.

To accelerate lookup, a high-frequency buffer caches recently accessed exemplars. Entries in this buffer are ranked by a hybrid metric combining embedding similarity and hit frequency: $\text{Score}(E_j) = \lambda_{\text{sim}} \cos(e_{q_i}, e_{E_j}) + \lambda_{\text{hit}} \text{Count}(E_j)$, so that frequently used exemplars gain higher reuse priority. This dual-layer design

Table 1: Performance comparison on MultiTQ (Hits@1, %). Best in bold, second-best underlined.

Model	LLM	Overall	Question Type		Answer Type	
			Multiple	Single	Entity	Time
BERT[8]	–	8.3	6.1	9.2	10.1	4.0
ALBERT[21]		10.8	8.6	11.6	13.9	3.2
EmbedKGQA[37]	–	20.6	13.4	23.5	29.0	0.1
CronKGQA[36]		27.9	13.4	33.7	32.8	15.6
MultiQA[5]		29.3	15.9	34.7	34.9	15.7
ChatGPT	GPT-3.5-Turbo	10.2	7.7	14.7	13.7	2.0
KG-RAG[4]		18.5	16.0	20.0	23.0	7.0
ReAct KB[4]		21.0	13.6	63.5	31.3	30.0
ARI[4]		38.0	68.0	21.0	39.4	34.4
TempAgent[13]		53.9	16.8	68.4	47.8	66.1
MemoTime	GPT-4o-mini	64.2	40.3	73.0	61.5	70.8
	Qwen3-32B	68.2	42.5	77.6	62.1	81.7
	DeepSeek-V3	73.0	45.9	82.9	67.7	84.6
	GPT-4-Turbo	77.9	53.8	86.8	74.5	85.3

allows MemoTime to balance long-term generalization with short-term adaptability during ongoing reasoning.

Experience update. After a reasoning cycle concludes, all verified sub-questions, selected toolkits, and reasoning paths are recorded back into $\mathcal{E}_{pool}$. Each record includes textual content, structured indicators, execution parameters, temporal constraints, and corresponding embeddings for both the question and the indicator. This dual representation supports precise semantic alignment and fast vector retrieval. New entries are inserted into both the global memory and the buffer; obsolete or low-value traces are periodically pruned to prevent redundancy. Through continuous recording and pruning, the experience pool evolves alongside the system’s reasoning behaviour, serving as a compact yet expressive representation of accumulated knowledge.

Cross-type augmentation. While retrieval is type-restricted, the memory layer supports cross-type augmentation during updates. When a sub-question from one temporal type exhibits high structural similarity with exemplars in another, the new record is annotated with multiple secondary type labels. This mechanism enables MemoTime to share transferable reasoning patterns across related temporal operators (e.g., BeforeLast and AfterFirst), forming an interconnected experience graph that improves recall and diversity without sacrificing retrieval precision.

Continual adaptation. The memory layer continuously re-weights and reorganizes stored exemplars based on retrieval frequency, temporal freshness, and reasoning success. High-confidence exemplars are prioritized in future retrievals, while outdated or inconsistent examples gradually decay in influence. The embedding index is periodically rebalanced to maintain uniform coverage and retrieval efficiency. This self-adaptive mechanism ensures that MemoTime focuses on temporally relevant reasoning strategies and generalizes effectively to unseen question types.

Integration with reasoning. The experience memory interacts closely with the Tree of Time reasoning loop described in Section 4.2. At runtime, it acts as a shared repository for experience-guided decomposition, toolkit selection, and answer synthesis. After each reasoning session, verified results, including embeddings, type labels, sufficiency status, and hit statistics, are propagated back into memory, updating both the long-term index and the

Table 2: Performance comparison on TimeQuestions (Hits@1, %). Best in bold, second-best underlined.

Model	LLM	Overall	Question Type		Answer Type	
			Explicit	Implicit	Temporal	Ordinal
PullNet[42]	–	10.5	2.2	8.1	23.4	2.9
Uniqorn[34]		33.1	31.8	31.6	39.2	20.2
GRAFT-Net[41]		45.2	44.5	42.8	51.5	32.2
CronKGQA[28]	–	46.2	46.6	44.5	51.1	36.9
TempoQR[28]		41.6	46.5	3.6	40.0	34.9
EXAQT[18]		57.2	56.8	51.2	64.2	42.0
TwIRGCN[38]		60.5	60.2	58.6	64.1	51.8
LGQA[25]		52.9	53.2	50.6	60.5	40.2
GenTKGQA[11]	Fine-tune	58.4	59.6	<u>61.1</u>	56.3	<u>57.8</u>
TimeR ⁴ [35]	Fine-tune	64.8	<u>66.0</u>	52.9	77.6	45.5
ChatGPT	GPT-3.5-Turbo	45.9	43.3	51.1	46.5	48.1
MemoTime	GPT-4o-mini	60.9	60.0	50.0	69.4	52.8
	Qwen3-32B	60.6	58.3	51.1	69.8	53.3
	DeepSeek-V3	<u>67.8</u>	63.1	53.3	71.3	54.3
	GPT-4-Turbo	71.4	67.0	67.5	<u>74.5</u>	58.7

fast-access buffer. This design closes the reasoning–learning loop, enabling MemoTime to continually refine its temporal reasoning ability through iterative retrieval, execution, and self-improvement.

5 Experiments

In this section, we evaluate MemoTime on two challenge TKGQA datasets. The detailed experimental settings, including datasets, baselines, and implementations, can be found in Appendix D.

5.1 Main results

We evaluate MemoTime against a comprehensive set of baselines on MultiTQ and TimeQuestions, as shown in Table 1 and Table 2. On the MultiTQ dataset, traditional pre-trained language models (BERT, ALBERT) and embedding-based methods (EmbedKGQA, CronKGQA, MultiQA) show limited temporal reasoning ability, with overall accuracies below 30%. Their static embeddings fail to capture multi-hop or cross-entity temporal dependencies. LLM-based methods, e.g., KG-RAG, ReAct KB, ARI, TempAgent, leverage prompt reasoning but remain sensitive to implicit time expressions and struggle with multi-constraint temporal alignment. In contrast, MemoTime consistently outperforms all competitors across both question and answer types. MemoTime with GPT-4-Turbo achieves an overall 77.9% Hit@1, surpassing TempAgent by 24.0% and outperforming all GPT-3.5 baselines by a large margin. Even smaller backbones (e.g., Qwen3-32B) reach 68.2%, exceeding the best GPT-3.5-based models. These results confirm that MemoTime effectively integrates temporal grounding, hierarchical reasoning, and operator-aware retrieval to ensure temporal faithfulness and reasoning stability. On the TimeQuestions dataset, which emphasizes explicit versus implicit temporal expressions and temporal versus ordinal answer types, MemoTime again achieves the best overall results. As shown in Table 2, MemoTime with GPT-4-Turbo achieves 71.4% overall accuracy and 74.5% on temporal-type questions, outperforming the fine-tuned GenTKGQA and TimeR⁴ models, despite being fully training-free. This demonstrates MemoTime’s ability to handle implicit temporal relations and operator combinations through memory-guided decomposition and dynamic temporal retrieval. Overall, across both datasets, MemoTime achieves new state-of-the-art results under all backbone LLMs, validating its design as a memory-augmented temporal reasoning framework capable

Table 3: Performance comparison between the IO baseline and MemoTime across two temporal QA datasets using six backbone LLMs. The highest improvement is highlighted in bold, and the second-best is underlined for each data.

Dataset	Qwen3-4B			Qwen3-8B			Qwen3-32B			Qwen3-80B			DeepSeek-V3			GPT-4-Turbo		
	IO	MemoTime	↑	IO	MemoTime	↑	IO	MemoTime	↑	IO	MemoTime	↑	IO	MemoTime	↑	IO	MemoTime	↑
MultiTQ	3.5	55.3	14.8 ×	2.5	57.0	21.8 ×	1.3	61.4	46.2 ×	9.5	67.5	6.1 ×	7.5	70.9	8.5 ×	12.0	76.3	5.4 ×
TimeQuestion	18.0	45.3	1.5 ×	28.5	49.4	0.7 ×	30.0	<u>60.6</u>	1.0 ×	41.0	62.7	0.5 ×	43.5	64.6	0.5 ×	44.0	68.1	0.5 ×

Table 4: Ablation study on MultiTQ (Hits@1, %).

Model Variant	Overall	Question Type		Answer Type	
		Multiple	Single	Entity	Time
MemoTime (Full w/GPT-4o-mini)	64.2	40.3	73.0	61.5	70.8
w/o Graph-based Retrieval	52.9	23.5	65.0	48.2	64.2
w/o Embedding Retrieval	60.1	38.6	68.0	57.4	66.8
w/o Temporal Evidence Retrieval	11.2	8.0	12.4	11.5	12.0
w/o Question Tree	58.3	19.3	72.6	54.6	70.1
w/o Experience Memory	59.8	26.1	72.2	55.8	69.6

of learning from experience, synchronizing multiple entities, and maintaining temporal consistency across diverse question types.

5.2 Ablation Study

How does the performance of MemoTime vary across different LLM backbones? To evaluate the generality of our approach, we tested MemoTime on six LLM backbones, including Qwen3-4B, Qwen3-8B, Qwen3-32B, Qwen3-80B, DeepSeek-V3, and GPT-4-Turbo, across two temporal QA benchmarks (MultiTQ and TimeQuestions). As shown in Table 3, MemoTime consistently improves performance over the IO baseline across all settings. On the more complex MultiTQ, which requires multi-hop and cross-entity temporal reasoning, MemoTime boosts accuracy from as low as 1.3% to 61.4% on Qwen3-32B, achieving a 46.2 times relative gain. Even the smallest backbones (e.g., Qwen3-4B) experience a 14.8 times improvement, demonstrating that our recursive, memory-augmented reasoning compensates for weaker temporal understanding in smaller models. For TimeQuestions, which involves simpler, single-hop temporal relations, MemoTime still improves performance modestly by up to 1.5 times, indicating consistent stability across reasoning difficulty levels. Overall, MemoTime enables smaller models to perform competitively with large-scale LLMs such as GPT-4-Turbo, while stronger models continue to benefit from enhanced temporal faithfulness and reduced reasoning variance. These results confirm that MemoTime generalizes well across architectures and scales, serving as a plug-in temporal reasoning module rather than relying on model-specific fine-tuning.

How does temporal evidence retrieval affect performance? To assess the contribution of the temporal evidence retrieval module, we disable different retrieval components. As shown in Table 4, removing graph-based retrieval reduces the overall accuracy from 64.2% to 52.9%, with a substantial drop on multiple-entity questions (from 40.3% to 23.5%), indicating that explicit structural traversal is essential for maintaining temporally aligned entity paths. When embedding-based retrieval is removed, the performance decreases moderately from 64.2% to 60.1%, suggesting that semantic retrieval complements the graph search by capturing lexically diverse or contextually implicit temporal cues. In contrast, when all external retrieval modules are disabled and the model relies solely on its internal knowledge, the accuracy falls drastically to 11.2%, reflecting

a near-complete loss of temporal grounding. These findings confirm that temporal evidence retrieval is not only indispensable but also synergistic: graph-based retrieval ensures factual precision through topological consistency, while embedding-based retrieval enhances coverage through semantic generalization.

How does hierarchical decomposition affect performance?

To examine the role of hierarchical decomposition, we disable the Tree-of-Time module, forcing the model to reason over the full question without recursive planning. As shown in Table 4, the overall accuracy increases from 64.2% to 58.3%, while performance on multiple-entity questions drops sharply from 40.3% to 19.3%. This substantial decline demonstrates that recursive decomposition is critical for handling complex temporal dependencies and multi-hop constraints. Even with retrieval retained, the absence of decomposition limits the model’s ability to enforce monotonic timestamp ordering and to synchronize temporal operators across entities, leading to degraded reasoning coherence.

How does experience memory affect performance? To analyse the role of continual learning, we remove the experience memory module, preventing MemoTime from accessing previously verified reasoning traces. The result in Table 4 shows it causes a moderate performance drop from 64.2% to 59.8%, with larger reductions observed on multi-hop questions, from 40.3% to 26.1%. The results indicate that experience memory enhances reasoning stability and efficiency. Although the model remains competitive without it, the memory mechanism contributes to more consistent reasoning trajectories and progressive self-improvement across inference cycles.

To further evaluate the effectiveness and efficiency of MemoTime, we conduct additional experiments, including multi-granular temporal QA analysis (Appendix B.1), efficiency analysis (Appendix B.2), and specialized temporal retrieval toolkits (Appendix C).

6 Conclusion

In this paper, we present **MemoTime**, a memory-augmented temporal knowledge graph framework that enhances LLM temporal reasoning through structured grounding, hierarchical decomposition, and continual experience learning. MemoTime enables temporally faithful, multi-entity reasoning by constructing operator-aware temporal paths and dynamically reusing verified reasoning traces. Its adaptive retrieval and memory integration allow it to balance efficiency, accuracy, and temporal consistency across diverse reasoning scenarios. Extensive experiments on multiple temporal QA datasets demonstrate that MemoTime outperforms existing baselines, showcasing its superior reasoning capabilities and interoperability.

Acknowledgments

Xiaoyang Wang is supported by the Australian Research Council DP230101445 and DP240101322. Wenjie Zhang is supported by the Australian Research Council DP230101445 and FT210100303.

References

- [1] James F Allen. 1984. Towards a general theory of action and time. *Artificial intelligence* 23, 2 (1984), 123–154.
- [2] Tom B Brown. 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165* (2020).
- [3] Liyi Chen, Panrong Tong, et al. 2024. Plan-on-Graph: Self-Correcting Adaptive Planning of Large Language Model on Knowledge Graphs. In *NeurIPS*.
- [4] Ziyang Chen, Dongfang Li, Xiang Zhao, et al. 2024. Temporal knowledge question answering via abstract reasoning induction. In *ACL*.
- [5] Ziyang Chen, Jinzhi Liao, and Xiang Zhao. 2023. Multi-granularity temporal question answering over knowledge graphs. In *ACL*. 11378–11392.
- [6] Ziyang Chen, Jinzhi Liao, and Xiang Zhao. 2023. Multi-granularity Temporal Question Answering over Knowledge Graphs. In *ACL*. 11378–11392.
- [7] Zhuo Chen, Zhao Zhang, et al. 2024. Self-improvement programming for temporal knowledge graph question answering. *arXiv preprint arXiv:2404.01720* (2024).
- [8] Jacob Devlin, Ming-Wei Chang, et al. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*.
- [9] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, et al. 2024. The faiss library. *arXiv preprint arXiv:2401.08281* (2024).
- [10] Yair Feldman and Ran El-Yaniv. 2019. Multi-hop paragraph retrieval for open-domain question answering. *arXiv preprint arXiv:1906.06606* (2019).
- [11] Yifu Gao, Linbo Qiao, et al. 2024. Two-stage generative question answering on temporal knowledge graph using large language models. In *ACL*.
- [12] Xinyan Guan, Yanjiang Liu, et al. 2024. Mitigating large language model hallucinations via autonomous knowledge graph-based retrofitting. In *AAAI*.
- [13] Qianyi Hu, Xinhui Tu, et al. 2025. Time-aware ReAct Agent for Temporal Knowledge Graph Question Answering. In *Findings of NAACL*.
- [14] Chengkai Huang, Hongtao Huang, Tong Yu, et al. 2025. A Survey of Foundation Model-Powered Recommender Systems: From Feature-Based, Generative to Agentic Paradigms. *arXiv preprint arXiv:2504.16420* (2025).
- [15] Chengkai Huang, Yu Xia, Rui Wang, et al. 2025. Embedding-informed adaptive retrieval-augmented generation of large language models. In *COLING*.
- [16] Zhen Jia, Abdalghani Abujabal, et al. 2018. TEQUILA: Temporal Question Answering over Knowledge Bases. In *CIKM*.
- [17] Zhen Jia, Philipp Christmann, and Gerhard Weikum. 2024. Faithful temporal question answering over heterogeneous sources. In *WWW*. 2052–2063.
- [18] Zhen Jia, Soumajit Pramanik, Rishiraj Saha Roy, and Gerhard Weikum. 2021. Complex temporal question answering on knowledge graphs. In *CIKM*. 792–802.
- [19] Jinhao Jiang et al. 2023. Structgpt: A general framework for large language model to reason over structured data. *arXiv preprint arXiv:2305.09645* (2023).
- [20] Tushar Khot et al. 2022. Decomposed prompting: A modular approach for solving complex tasks. *arXiv preprint arXiv:2210.02406* (2022).
- [21] Zhenzhong Lan, Mingda Chen, et al. 2019. Albert: A lite bert for self-supervised learning of language representations. *arXiv preprint arXiv:1909.11942* (2019).
- [22] Fan Li, Xiaoyang Wang, Dawei Cheng, et al. 2025. Efficient dynamic attributed graph generation. In *ICDE*.
- [23] Fan Li, Zhiyu Xu, et al. 2024. AdaRisk: risk-adaptive deep reinforcement learning for vulnerable nodes detection. *IEEE TKDE* (2024).
- [24] Ke Liang, Lingyuan Meng, et al. 2023. Learn from relational correlations and periodic events for temporal knowledge graph reasoning. In *SIGIR*.
- [25] Yonghao Liu, Di Liang, Mengyu Li, et al. 2023. Local and Global: Temporal Question Answering via Information Fusion. In *IJCAI*. 5141–5149.
- [26] Linhao Luo et al. 2023. Reasoning on graphs: Faithful and interpretable large language model reasoning. *arXiv preprint arXiv:2310.01061* (2023).
- [27] Shengjie Ma, Chengjin Xu, et al. 2024. Think-on-Graph 2.0: Deep and Interpretable Large Language Model Reasoning with Knowledge Graph-guided Retrieval. *arXiv preprint arXiv:2407.10805* (2024).
- [28] Costas Mavromatis, Prasanna Lakshmi Subramanyam, et al. 2022. Tempoqr: temporal question reasoning over knowledge graphs. In *AAAI*.
- [29] Kai Mei, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. 2024. AIOS: LLM agent operating system. *arXiv e-prints*, pp. arXiv–2403 (2024).
- [30] Sewon Min, Victor Zhong, et al. 2019. Multi-hop reading comprehension through question decomposition and rescoring. *arXiv preprint arXiv:1906.02916* (2019).
- [31] Sumit Neelam and Udit others Sharma. 2022. SYGMA: A System for Generalizable and Modular Question Answering Over Knowledge Bases. In *Findings of EMNLP*.
- [32] Shirui Pan, Linhao Luo, et al. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE TKDE* (2024).
- [33] Fabio Petroni et al. 2020. KILT: a benchmark for knowledge intensive language tasks. *arXiv preprint arXiv:2009.02252* (2020).
- [34] Soumajit Pramanik et al. 2024. Uniqorn: unified question answering over rdf knowledge graphs and natural language text. *Journal of Web Semantics* (2024).
- [35] Xinying Qian et al. 2024. TimeR⁴: Time-aware Retrieval-Augmented Large Language Models for Temporal Knowledge Graph Question Answering. In *EMNLP*.
- [36] Apoorv Saxena, Soumen Chakrabarti, et al. 2021. Question answering over temporal knowledge graphs. *arXiv preprint arXiv:2106.01515* (2021).
- [37] Apoorv Saxena, Aditay Tripathi, et al. 2020. Improving multi-hop question answering over knowledge graphs using knowledge base embeddings. In *ACL*.
- [38] Aditya Sharma et al. 2022. Twirgcn: Temporally weighted graph convolution for question answering over temporal knowledge graphs. *arXiv preprint arXiv:2210.06281* (2022).
- [39] Qing Sima, Xiaoyang Wang, et al. 2026. Beyond Homophily: Community Search on Heterophilic Graphs. *arXiv preprint arXiv:2601.01703* (2026).
- [40] Qing Sima, Jianke Yu, et al. 2025. Deep overlapping community search via subspace embedding. *SIGMOD* (2025).
- [41] Haitian Sun et al. 2018. Open domain question answering using early fusion of knowledge bases and text. *arXiv preprint arXiv:1809.00782* (2018).
- [42] Haitian Sun et al. 2019. Pullnet: Open domain question answering with iterative retrieval on knowledge bases and text. *arXiv preprint arXiv:1904.09537* (2019).
- [43] Jiashuo Sun, Chengjin Xu, et al. 2024. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. In *ICLR*.
- [44] Qiang Sun, Sirui Li, et al. 2025. TimelineKGQA: A Comprehensive Question-Answer Pair Generator for Temporal Knowledge Graphs. In *WWW*.
- [45] Alon Talmor et al. 2018. Commonsenseqa: A question answering challenge targeting commonsense knowledge. *arXiv preprint arXiv:1811.00937* (2018).
- [46] Xingyu Tan, Jingya Qian, Chen Chen, Sima Qing, Yanping Wu, Xiaoyang Wang, and Wenjie Zhang. 2023. Higher-order peak decomposition. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*.
- [47] Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, and Wenjie Zhang. 2025. Paths-over-graph: Knowledge graph empowered large language model reasoning. In *Proceedings of the ACM on Web Conference 2025*. 3505–3522.
- [48] Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, Liming Zhu, and Wenjie Zhang. 2025. Hydrarag: Structured cross-source enhanced large language model reasoning. In *EMNLP*. 14442–14470.
- [49] Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, Liming Zhu, and Wenjie Zhang. 2026. PrivGemo: Privacy-Preserving Dual-Tower Graph Retrieval for Empowering LLM Reasoning with Memory Augmentation. *arXiv preprint arXiv:2601.08739* (2026).
- [50] Bing Wang et al. 2023. Enhancing large language model with self-controlled memory framework. *arXiv preprint arXiv:2304.13343* (2023).
- [51] Jinghao Wang, Yanping Wu, Xiaoyang Wang, et al. 2025. Effective Influence Maximization with Priority. In *WWW*.
- [52] Jinghao Wang, Yanping Wu, Xiaoyang Wang, et al. 2025. Time-Critical Influence Minimization via Node Blocking. *SIGMOD* (2025).
- [53] Jinghao Wang, Yanping Wu, Xiaoyang Wang, Ying Zhang, et al. 2024. Efficient influence minimization via node blocking. *arXiv preprint arXiv:2405.12871* (2024).
- [54] Yiqi Wang, Long Yuan, et al. 2023. Towards efficient shortest path counting on billion-scale graphs. In *ICDE*.
- [55] Yiqi Wang, Long Yuan, et al. 2024. Simpler is more: Efficient top-k nearest neighbors search on large road networks. *arXiv preprint arXiv:2408.05432* (2024).
- [56] Jian Wu et al. 2024. Gendec: A robust generative question-decomposition method for multi-hop reasoning. *arXiv preprint arXiv:2402.11166* (2024).
- [57] Yanping Wu, Renjie Sun, Xiaoyang Wang, et al. 2024. Efficient Maximal Frequent Group Enumeration in Temporal Bipartite Graphs. *Proc. VLDB Endow.* (2024).
- [58] Yuhua Wu, Yuanyuan Xu, Xuemin Lin, and Wenjie Zhang. 2023. A holistic approach for answering logical queries on knowledge graphs. In *ICDE*.
- [59] Peiting Xie, Xiangjun Zai, Yanping Wu, et al. 2025. HL-index: Fast Reachability Query in Hypergraphs. *arXiv preprint arXiv:2512.23345* (2025).
- [60] Zhengyi Yang, Wenjie Zhang, Xuemin Lin, et al. 2023. Hgmatch: A match-by-hyperedge approach for subgraph matching on hypergraphs. In *ICDE*.
- [61] Shunyu Yao et al. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629* (2022).
- [62] Xiangjun Zai, Xingyu Tan, Xiaoyang Wang, Qing Liu, et al. 2025. PRoH: Dynamic Planning and Reasoning over Knowledge Hypergraphs for Retrieval-Augmented Generation. *arXiv preprint arXiv:2510.12434* (2025).
- [63] Zian Zhai, Fan Li, Xingyu Tan, Xiaoyang Wang, and Wenjie Zhang. 2025. Graph is a natural regularization: Revisiting vector quantization for graph representation learning. *arXiv preprint arXiv:2508.06588* (2025).
- [64] Zian Zhai, Qing Sima, Xiaoyang Wang, and Wenjie Zhang. 2025. SGPT: Few-Shot Prompt Tuning for Signed Graphs. In *CIKM*.
- [65] Hang Zhang et al. 2021. Poolingformer: Long document modeling with pooling attention. In *ICML*. PMLR.
- [66] Tianming Zhang, Xinwei Cai, et al. 2024. Towards efficient simulation-based constrained temporal graph pattern matching. *World Wide Web* (2024).
- [67] Tianming Zhang, Junkai Fang, et al. 2024. Tatk: A temporal graph neural network for fast approximate temporal Katz centrality ranking. In *WWW*.
- [68] Liangwei Nathan Zheng, Chang George Dong, et al. 2024. Understanding Why Large Language Models Can Be Ineffective in Time Series Analysis: The Impact of Modality Alignment. *arXiv preprint arXiv:2410.12326* (2024).
- [69] Liangwei Nathan Zheng, Wenhao Liang, Wei Emma Zhang, et al. 2025. Lifting Manifolds to Mitigate Pseudo-Alignment in LLM4TS. *arXiv preprint arXiv:2510.12847* (2025).
- [70] Wanjuan Zhong, Lianhong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. Memorybank: Enhancing large language models with long-term memory. In *AAAI*.

A Algorithm

A.1 Temporal Grounding

We present the comprehensive algorithmic procedure for temporal grounding (Section 4.1) in Algorithm 1.

A.2 Tree of Time Reasoning

We present the comprehensive algorithmic procedure for Tree of Time hierarchical reasoning (Section 4.2) in Algorithm 2.

A.3 Temporal Evidence Retrieval and Pruning

We present the comprehensive algorithmic procedure for temporal evidence retrieval and pruning (Section 4.3) in Algorithm 3.

A.4 Experience Memory

We present the comprehensive algorithmic procedure for experience memory (Section 4.4) in Algorithm 4.

Algorithm 1: TemporalGrounding

Input : Question Q , TKG $\mathcal{G}$, experience pool $\mathcal{E}_{pool}$, limits D_{max}, W_{exp}
Output : Subquestion tree $\mathcal{T}_Q$, indicators $\{I_i\}$, topic entities $Topics$, type $Type(Q)$, subgraph $\mathcal{G}_Q$
 /* LLM extraction + DRM alignment */
 1 $Keywords \leftarrow NER(Q); Cand \leftarrow LinkToKG(Keywords, \mathcal{G});$
 2 $Topics \leftarrow Disambiguate(Q, Cand);$
 /* bounded D_{max} -hop temporal neighborhood */
 3 $\mathcal{G}_Q \leftarrow BuildTemporalSubgraph(Topics, \mathcal{G}, D_{max});$
 /* Temporal type classification */
 4 $\mathcal{E}_{type} \leftarrow GetTypeExp(\mathcal{E}_{pool}, Q, W_{exp});$
 5 $Type(Q) \leftarrow Prompt_{TypeSelect}(Q, \mathcal{E}_{type});$
 6 **Return** $\mathcal{T}_Q, \{I_i\}, Topics, Type(Q), \mathcal{G}_Q;$

Algorithm 2: TreeofTimeReasoning

Input : indicators $\{I_i\}$, subgraph $\mathcal{G}_Q$, topics $Topics$, experience pool $\mathcal{E}_{pool}$, limits $D_{max}, B_{max}, W_{max}, W_{exp}$
Output : Local answers/paths per node; updated tree
 1 $\mathcal{E}_{decomp} \leftarrow GetDecompExp(\mathcal{E}_{pool}, Q, W_{exp}, Type(Q));$
 2 **if** $\exists$ compatible plan in $\mathcal{E}_{decomp}$ **then**
 3 $\mathcal{T}_Q \leftarrow ReusePlan(\mathcal{E}_{decomp}, Q, Topics);$
 4 **else** $\mathcal{T}_Q \leftarrow Prompt_{Decompose}(Q, \mathcal{E}_{decomp}, Type(Q));$
 5 $\{I_i\} \leftarrow ExtractIndicators(\mathcal{T}_Q);$
 6 **for each** $(q_i, I_i) \in TraverseRootToLeaf(\mathcal{T}_Q)$ **do**
 7 $(\hat{a}, \hat{p}, sufficient) \leftarrow MemoryLookupAndTest(q_i, I_i, \mathcal{E}_{pool});$
 8 **if** sufficient **then continue**;
 9 $\mathcal{E}_{tool} \leftarrow GetToolkitExp(\mathcal{E}_{pool}, q_i, I_i, Type(Q), W_{exp});$
 10 $\mathcal{T}_i \leftarrow Prompt_{ToolkitSelect}(\mathcal{T}_Q, I_i, q_i, \mathcal{E}_{tool});$
 11 $(C_i, S_i) \leftarrow$
 TemporalRetrieveAndPrune($\mathcal{G}_Q, q_i, I_i, Topics, \mathcal{T}_i, \mathcal{E}_{pool}, W_{max}$);
 $(a_i, P_i, sufficient) \leftarrow DebateVoteAndTest(q_i, I_i, C_i);$
 12 **if** $\neg$ sufficient **then**
 13 **if** DepthOK $\wedge$ BranchOK **then**
 14 $\{q_{new}\}, \{I_{new}\} \leftarrow RefineOrDecompose(q_i, I_i, C_i);$
 15 UpdateTree($\mathcal{T}_Q, \{q_{new}\}, \{I_{new}\}$);
 16 **else** WriteBackIfSuccess($\mathcal{E}_{pool}, q_i, I_i, a_i, P_i, \mathcal{T}_Q$);
 17 **Return** $\{a_i\}, \{P_i\}, \mathcal{T}_Q;$

Algorithm 3: TemporalRetrieveAndPrune

Input : Subquestion (q_i, I_i) , topics $Topics$, selected toolkits $\mathcal{T}_i$, experience pool $\mathcal{E}_{pool}$, subgraph $\mathcal{G}_Q$, limits $W_{max}, W_{exp}, D_{max}$
Output : Pruned candidates C_i , selected seeds S_i
 1 $\mathcal{E}_{seed} \leftarrow GetSeedExp(\mathcal{E}_{pool}, q_i, I_i, W_{exp});$
 2 $S_i \leftarrow Prompt_{SeedSelect}(Topics, I_i, q_i, \mathcal{E}_{seed}); C \leftarrow \emptyset;$
 3 **for each** $T_\theta \in \mathcal{T}_i$ **in parallel do**
 4 $C \leftarrow C \cup TreeBasedTemporalPathRetrieval(\mathcal{G}_Q, T_\theta, I_i, S_i, D_{max});$
 5 $C \leftarrow C \cup DenseEmbedRetrieve(I_i, T_\theta, Docu(\mathcal{G}_Q));$
 6 $\tilde{C} \leftarrow \{p \in C \mid p \models C_{time}(I_i) \wedge Monotone(p)\};$
 7 **for each** $p \in \tilde{C}$ **do**
 8 $s_{sem}(p) \leftarrow SemanticDRM(I_i, p);$
 9 $s_{prox}(p) \leftarrow \exp(-|t(p) - t(I_i)|/\sigma);$
 10 $Score(p) \leftarrow \lambda_{sem}s_{sem}(p) + \lambda_{prox}s_{prox}(p);$
 11 $\tilde{C} \leftarrow SelectTopPathandSort(\tilde{C}, W_i);$
 12 $C_i \leftarrow Prompt_{SelectPath}(\tilde{C}, q_i, I_i, W_{max});$
 13 **Return** $C_i, S_i;$
 14 **Procedure** TreeBasedTemporalPathRetrieval($\mathcal{G}_Q, T_\theta, I_i, S_i, D_{max}$)
 15 StartList, $C_{time}, D_{pred} \leftarrow ExtractInfo(T_\theta, I_i, S_i);$
 16 $D \leftarrow 1; Paths \leftarrow \emptyset;$
 Frontier $\leftarrow \{(s, t_0, []) \mid s \in StartList, t_0 = InitTime(C_{time})\};$
 17 **while** $D \leq D_{max}$ **do**
 18 NewFrontier $\leftarrow \emptyset;$
 19 **for each** $(v, t_{last}, P) \in Frontier$ **do**
 20 **for each** $(v \xrightarrow{r, te} u) \in ExpandOneHop(\mathcal{G}_Q, v, T_\theta)$ **do**
 21 **if** $\neg Monotone(t_{last}, t_e)$ **then**
 22 **continue**
 23 $P' \leftarrow P \parallel (v \xrightarrow{r, te} u); Paths \leftarrow Paths \cup \{P'\};$
 24 NewFrontier $\leftarrow NewFrontier \cup \{(u, t_e, P')\};$
 25 **if** $|NewFrontier| > W_{max}$ **then**
 26 NewFrontier $\leftarrow RelevantPruning(NewFrontier, I_i, W_{max});$
 27 Frontier $\leftarrow NewFrontier; D \leftarrow D+1;$
 28 **Return** Paths;

Algorithm 4: Experience Memory

Input : Experience pool $\mathcal{E}_{pool}$, query object (question or sub-question), indicator I , type label τ , buffer size K
Output : Matched exemplars $\mathcal{E}_{match}$ (optional), updated pool
 1 $e_q \leftarrow Encode(q); e_l \leftarrow Encode(I);$
 2 $\mathcal{E}_{match} \leftarrow ANN_Search(\mathcal{E}_{pool}, [e_q, e_l], \text{filter: type} = \tau, \text{top-}K);$
 3 RankBy($\lambda_{sim} \cdot sim + \lambda_{hit} \cdot hit_count$);
 4 **Return** $\mathcal{E}_{match};$
 5 **Procedure** WriteBackIfSuccess($\mathcal{E}_{pool}, q, I, a, P$)
 6 Store(q, I, a, P, τ , embeddings = $[e_q, e_l]$, sufficient = true);
 7 UpdateBufferStats(q, I);
 8 **Procedure** MemoryLookupAndTest($q, I, \mathcal{E}_{pool}$)
 9 Hist $\leftarrow Retrieve(q, I, \text{filter: type} = \tau);$
 10 **for each** $(\hat{a}, \hat{p}) \in Hist$ **do**
 11 **if** Sufficient($q, I, \hat{a}, \hat{p}$) **then Return** $\hat{a}, \hat{p}, \text{sufficient} = \text{true};$
 12 **Return** $\perp, \perp, \text{sufficient} = \text{false};$

Table 5: Multi-granularity temporal reasoning on Hits@1. Best in each column is in bold.

Model	Equal			Before/After			Equal-Multi		
	Day	Month	Year	Day	Month	Year	Day	Month	Year
BERT	4.9	10.3	13.6	15.0	16.4	17.5	6.4	10.2	9.0
DistillBERT	4.1	8.7	11.3	16.0	15.0	18.6	9.6	12.7	8.9
ALBERT	6.9	8.2	13.2	22.1	27.7	30.8	10.3	14.4	14.4
EmbedKGQA	20.0	33.6	21.8	39.2	51.8	51.1	14.5	32.1	26.3
CronKGQA	42.5	38.9	33.1	37.5	47.4	45.0	29.5	33.3	25.1
MultiQA	44.5	39.3	35.0	37.9	54.8	52.5	30.8	32.1	28.3
MemoTime w/ DeepSeek-V3	85.6	85.1	92.3	81.4	77.4	82.4	60.0	55.3	53.2
MemoTime w/ GPT-4-Turbo	85.8	85.0	93.8	92.5	86.2	94.1	68.4	70.4	64.4

Table 6: Efficiency analysis of MemoTime on MultiTQ.

	Overall	After-First	Before-Last	Equal-Multi	First/Last	Before/After	Equal
Average Depth	1.37	2.12	2.00	1.57	1.00	1.54	1.04
Average Branch	2.64	3.91	3.94	3.05	2.00	2.78	2.01
Average LLM Calls	8.75	11.55	11.49	9.88	7.42	9.10	7.32
Running Time (s)	43.11	58.11	57.89	48.86	35.70	45.34	35.42

B Additional Experiments

B.1 Multi-Granular Temporal QA Analysis

We further evaluate MemoTime on temporal reasoning tasks that require distinguishing and aligning facts across different temporal granularities, including day, month, and year levels. This setting examines whether models can adapt to diverse temporal resolutions and maintain consistent reasoning performance. As shown in Table 5, traditional pre-trained and embedding-based baselines exhibit significant degradation as the temporal resolution becomes finer or when multiple granularities are combined. For example, while MultiQA achieves 44.5% accuracy on the Equal-Day level, its performance drops sharply to 28.3% on Equal-Multi-Year, indicating that these models struggle to maintain chronological consistency when reasoning across mixed time scales.

In contrast, MemoTime consistently achieves strong and stable results across all granularities and operators, reaching up to 94.1% Hits@1 on Before/After-Year and maintaining 70.4% under the most complex Equal-Multi-Month setting. The consistent improvement across both fine-grained and compound temporal tasks demonstrates that MemoTime effectively adapts to the diversity of temporal reasoning requirements. This robustness is mainly attributed to two key components: (1) operator-aware decomposition, which dynamically adjusts reasoning depth and retrieval scope according to the temporal operator, and (2) temporal path construction, which enforces monotonic timestamp progression and co-aligns evidence across multiple resolutions. Together, these mechanisms enable unified and faithful temporal reasoning while preserving interpretability across heterogeneous granularities.

B.2 Efficiency Analysis

To comprehensively evaluate the efficiency of MemoTime, we conduct three analyses on the MultiTQ dataset: LLM calls cost analysis, running time analysis, and computational cost analysis. Table 6

presents the averaged statistics across different question types, providing insights into reasoning efficiency and structural scalability.

LLM calls cost analysis. To measure the efficiency of utilizing LLMs, we analyze the average number of LLM calls required to complete a reasoning cycle across different question types. As shown in Table 6, MemoTime performs an average of 8.75 calls per question, demonstrating its lightweight decomposition and reasoning pipeline. For simple operator types such as Equal and First/Last, the number of calls remains low (7–7.5), corresponding to a single reasoning round without recursive decomposition. In contrast, complex composite operators including After-First and Before-Last require deeper temporal validation, resulting in approximately 11.5 LLM calls. These results demonstrate that MemoTime maintains efficient reasoning, even for multi-hop and temporally constrained queries, by avoiding redundant invocations through structured decomposition and memory reuse.

Running time analysis. To further assess inference efficiency, we examine the average running time per question for different temporal types. The overall average runtime of MemoTime is 43.11 seconds per question, indicating an effective balance between retrieval and reasoning processes. Operator-specific analysis shows that single-hop or non-nested operators such as Equal and First/Last complete within approximately 35 seconds, whereas multi-hop operators like After-First and Before-Last require additional retrieval and temporal alignment, increasing the runtime to around 58 seconds. Despite these additional steps, MemoTime preserves scalability by dynamically pruning irrelevant temporal paths and parallelizing evidence collection within each hierarchical layer.

Computational cost analysis. To evaluate the structural efficiency of temporal reasoning, we analyze the average recursion depth and branching factor, which jointly characterize the complexity of decomposition and search. MemoTime exhibits a compact reasoning structure, with an average recursion depth of 1.37 and

Table 7: MemoTime toolkit library with purposes, key parameters, and operator mappings.

Toolkit	Purpose	Key parameters	Operator mapping
OneHop	One-hop neighbors with temporal filters	entity, direction, after/before, limit	seed, local
AfterFirst	First Nth event after a cutoff	entity, after, relation_filter, limit=N	after_first
BeforeLast	Last Nth event before a cutoff	entity, before, relation_filter, limit=N	before_last
BetweenRange	Events within a time window	entity, between=(start,end), granularity	during, between
DayEvents	Global events on a specific date	date, relation_filter, limit	same-day, snapshot
Month/YearEvents	Global events in a month/year	month or year, relation_filter, limit	same-month, same-year
DirectConnection	Direct edges between two entities	entity1, entity2, direction, time filters	pairwise, validate
Timeline	Chronological sequence for an entity	entity, direction, after/before, limit	ordering, stitch

branching factor of 2.64, indicating efficient but expressive reasoning behavior. As shown in Table 6, composite operators such as After-First and Before-Last require deeper reasoning hierarchies (depth around 2) and broader exploration (branch factor close to 4), whereas simpler operators such as Equal and First/Last remain shallow and nearly linear (depth near 1, branch factor around 2). These results demonstrate that MemoTime dynamically adjusts its computational footprint according to the temporal operator’s semantic complexity, achieving consistent temporal alignment while maintaining efficient reasoning performance.

C Toolkit Library

We implement eight specialized temporal retrieval toolkits exposing a unified interface with normalized parameters. Table 7 outlines their core purposes, key parameters, and corresponding temporal operators. The tools are composable and operator-aware under temporal constraints. They cover the majority of temporal reasoning patterns required for multi-hop question answering.

D Experiment Details

Experiment Datasets. Previous studies have shown that the CronQuestions dataset [30] contains spurious correlations that models can exploit to achieve inflated accuracy [38]. Following [35], we therefore evaluate MemoTime on two more challenging and widely used temporal QA benchmarks: MULTITQ[6] and TimeQuestions[18]. MULTITQ is the largest available TKGQA dataset, constructed from the ICEWS05-15 event database, containing over 500K unique question-answer pairs. It features multiple temporal granularities (year, month, and day), with questions spanning over 3,600 days and covering diverse temporal operators such as before, after, and equal. TimeQuestions is a smaller yet more linguistically complex benchmark comprising 16K temporal questions divided into four categories. Unlike MultiTQ, it uses only yearly granularity and focuses on questions that involve reasoning over alignment and temporal order rather than precise timestamps.

Baselines. We compare MemoTime against a comprehensive set of baselines. On **MultiTQ**, we include: (1) Pre-trained language models (PLMs) such as BERT [8] and ALBERT [21], which are fine-tuned for QA over temporal contexts; (2) Embedding-based temporal KGQA methods, including EmbedKGQA [37], CronKGQA [36], and

MultiQA [5], which leverage temporal embeddings to align question representations with time-stamped entity triples; (3) LLM-based in-context learning (ICL) approaches, such as direct instruction prompting (IO), KG-RAG [4], ReAct-KB [4, 61], ARI [4], and TempAgent [13], which combine TKG and reasoning via ChatGPT or similar models. On **TimeQuestions**, we evaluate: (1) Static KGQA baselines, including PullNet [42], UniQorn [34], and GRAFT-Net [41], which operate on non-temporal knowledge graphs; (2) Temporal KGQA baselines, including CronKGQA [36], TempoQR [28], EXAQT [18], and LGQA [25]; (3) LLM-based methods, including instruction prompting (IO) and two fine-tuned reasoning models: GenTKGQA [11], which fine-tunes TKG with LLaMA2-7B, and Timer⁴ [35], which couples a jointly fine-tuned retriever with ChatGPT reasoning. For each baseline, we adopt the reported performance values from their original publications to ensure comparability under consistent evaluation settings. Following [27, 43, 47], we use exact match accuracy (Hits@1) as the principal evaluation metric. Recall and F1 scores are not used since knowledge sources are not limited to document databases [27, 43].

Experiment Implementation. All experiments are conducted using GPT-4-Turbo as the primary reasoning backbone for MemoTime. To validate the generality and plug-and-play adaptability of our framework, we further instantiate MemoTime with several alternative LLMs, including DeepSeek-V3, Qwen3-Next-80B-A3B-Instruct (Qwen3-80B), Qwen3-32B, Qwen3-8B, and Qwen3-4B. These models represent a diverse spectrum of capacities and architectural scales, allowing us to evaluate how MemoTime performs under different model sizes and decoding behaviors. Following prior work [43, 47], the temperature is set to 0.4 during the evidence exploration phase to encourage reasoning diversity, and to 0 during the final answer generation stage to ensure deterministic output. The maximum generation length is 256 tokens. SentenceBERT [8] is utilized as dense retrieval module (DRM). For temporal path construction, we set $W_{\max} = 3$, $D_{\max} = 3$, $W_1 = 80$, and $\lambda_{\text{sem}} = 0.6$. During the experience memory phase, we use $\lambda_{\text{sim}} = 0.6$, $\lambda_{\text{hit}} = 0.4$, and $W_{\text{exp}} = 10$. All experiences are dynamically learned from verified reasoning traces, with five predefined exemplars used for cold-start initialization. The database buffer size is set to 200 for efficiency. The complete code implementation of MemoTime is publicly available.¹

¹<https://github.com/SteveTANTAN/Memotime>.